Devoir surveillé d'informatique

Consignes

- On pourra toujours librement utiliser une fonction demandée à une question précédente même si cette question n'a pas été traitée.
- Veillez à présenter vos idées et vos réponses partielles même si vous ne trouvez pas la solution complète à une question.
- La clarté et la lisibilité de la rédaction et des programmes sont des éléments de notation.

❑ Exercice 1 : *Anagrammes*

Deux mots *de même longueur* sont anagrammes l'un de l'autre lorsque l'un est formé en réarrangeant les lettres de l'autre. Par exemple :

— *niche* et *chien* sont des anagrammes.

— *epele* et *pelle* ne sont pas des anagrammes. En effet, bien qu'ils soient formés avec les mêmes lettres, la lettre *l* ne figure qu'à un seul exemplaire dans *epele* et il en faut deux pour écrire *pelle*.

Le but de l'exercice est d'écrire une fonction `anagrammes` qui prend en argument deux chaînes de caractères et qui renvoie `True` si ces deux chaînes sont des anagrammes et `False` sinon.

■ Partie I : Sans utilisation de dictionnaires

Q1– Écrire une fonction `compte_occ` qui prend en argument une chaîne de caractères `chaine` et un caractère `car` et renvoie le nombre d'apparitions de `car` dans `chaine`.

```
def compte_occ(chaine, car):
    cpt = 0
    for c in chaine:
        if c==car:
            cpt += 1
    return cpt
```

Q2– Montrer à l'aide d'un test approprié que la fonction `anagramme_bug` ci-dessous n'est pas correcte, c'est-à-dire qu'elle peut renvoyer `True` alors que les deux chaînes passées en argument ne sont pas des anagrammes.

```
def anagramme_bug(chaine1, chaine2):
    '''Renvoie True ssi chaine1 et chaine2 sont des anagrammes'''
    for c in chaine1:
        if compte_occ(chaine1,c)!=compte_occ(chaine2, c):
            return False
    return True
```

La fonction teste si tous les caractères de `chaine1` sont bien présents dans `chaine2` avec le bon nombre d'occurrences. En revanche, elle ne vérifie pas que ceux de `chaine2` sont dans `chaine1`. Par conséquent, elle renvoie `True` sur le test `anagramme_bug("erreur", "erreurs")`.

Q3– Corriger la fonction précédente afin de la rendre conforme à sa spécification.

```
def anagramme(chaine1, chaine2):
    '''Renvoie True ssi chaine1 et chaine2 sont des anagrammes'''
    for c in chaine1:
        if compte_occ(chaine1,c)!=compte_occ(chaine2, c):
            return False
    return len(chaine1)==len(chaine2)
```

■ **Partie II** : Avec les dictionnaires

On souhaite écrire une fonction `cree_dico` qui prend en argument une chaîne de caractères et renvoie un dictionnaire dont les clés sont les caractères composant la chaîne et les valeurs leur nombre d'apparitions.
Par exemple, `cree_dico("epele")` renvoie le dictionnaire `{'e': 3, 'p': 1, 'l': 1 }`. En effet, dans le mot `'epele'`, `'e'` apparaît à trois reprises et `'l'` et `'p'` chacun une fois.

Q4– Expliquer brièvement pourquoi la solution proposée ci-dessous (qui utilise la fonction `compte_occ` de la partie précédente) est de complexité quadratique en fonction de la taille n de la chaîne fournie en argument.

```
def cree_dico_quadratique(chaine):
    dico = {}
    for c in chaine:
        if c not in dico:
            dico[c] = compte_occ(chaine, c)
    return dico
```

Q5– Écrire une version de complexité linéaire de la fonction `cree_dico` puis expliquer brièvement pour quelle raison on obtient cette meilleure complexité.

```
def cree_dico(chaine):
    dico = {}
    for c in chaine:
        if c in dico:
            # caractère déjà présent, on ajoute 1 à son nombre d'occurrences
            dico[c] = dico[c]+1
        else:
            # caractère qui apparaît pour la première fois
            dico[c] = 1
    return dico
```

Le test d'appartenance à un dictionnaire étant de complexité $\mathcal{O}(1)$, la boucle ne contient que des opérations s'effectuant en temps constant et comme elle est parcourue n fois où n est la longueur de la chaîne, la complexité est bien linéaire.

Q6– Écrire une fonction `egaux` qui prend en argument deux dictionnaires et renvoie `True` si ces deux dictionnaires sont égaux (c'est-à-dire contiennent exactement les mêmes clés avec les mêmes valeurs) et `False` sinon.
Par exemple, `egaux({'e': 3, 'p': 1, 'l': 1 }, {'p': 1, 'e': 2, 'l': 2 })` renvoie `False`.
⚠ On s'interdit ici d'utiliser le test d'égalité `==` entre deux dictionnaires et on écrira un parcours de dictionnaire.

```
def egaux(dico1,dico2):
    if len(dico1)!=len(dico2):
        return False
    for cle in dico1:
        if (cle not in dico2 or dico1[cle]!=dico2[cle]):
            return False
    return True
```

Q7– Écrire une fonction `anagrammes_dico` qui prend en argument deux chaînes de caractères et renvoie `True` si ce sont des anagrammes et `False` sinon.

```python
def anagrammes_dico(chaine1,chaine2):
    dico1 = cree_dico(chaine1)
    dico2 = cree_dico(chaine2)
    return egaux(dico1,dico2)
```

❑ **Exercice 2** : *Requêtes SQL sur une seule table*

On considère la base de données `pays_du_monde` contenant une seule table `pays` dont le schéma est donné ci-dessous :

pays		
`nom`	:	TEXT
`region`	:	TEXT
`population`	:	INT
`surface`	:	INT
`cotes`	:	INT
`pib`	:	INT

D'autre part, on précise la signification des champs suivants :

- `population` : le nombre d'habitants du pays ;
- `region` : la région du pays (par exemple « Europe de l'ouest ») ;
- `surface` : la surface du pays (en km^2) ;
- `cotes` : la longueur côtière du pays, cette valeur vaut 0 lorsque le pays n'a pas d'ouverture sur la mer ;
- `pib` : le produit intérieur brut par habitant, c'est une mesure de la richesse du pays.

Et on indique que la requête `SELECT DISTINCT region FROM pays` a renvoyé le résultat suivant :

region
Asie
Afrique du nord
Europe de l'est
Europe de l'ouest
Océanie
Afrique subsaharienne
Proche orient
Amérique latine
Amérique du nord

Écrire les requêtes permettant de :

Q8– Trouver la population et le produit intérieur brut de la France.

```sql
SELECT population, pib
FROM pays
WHERE nom = "France";
```

Q9– Trouver les pays d'Europe n'ayant pas d'ouverture sur la mer.

```sql
SELECT nom
FROM pays
WHERE cotes=0 and (region="Europe de l'est" or region = "Europe de l'ouest");
```

Q10– Classer par ordre alphabétique les pays d'Amérique latine.

```sql
SELECT nom
FROM pays
WHERE region="Amérique latine"
ORDER BY nom;
```

Q11– Lister par ordre décroissant du nombre d'habitants les cinq pays les plus peuplés.

```sql
SELECT nom
FROM pays
ORDER BY population DESC
LIMIT 5;
```

Q12– Trouver le pays du Proche-Orient le plus riche (ayant le `pib` le plus élevé).

```sql
SELECT nom, MAX(pib)
FROM pays
WHERE region="Proche orient";
```

Q13– Classer les pays d'Afrique du Nord par densité décroissante de population (la densité est le rapport entre le nombre d'habitants et la surface).

```sql
SELECT nom, population/surface AS densite
FROM pays
WHERE region="Afrique du nord"
ORDER BY densite DESC;
```

Q14– Classer les régions par somme décroissante du PIB des pays qui les composent.

```sql
SELECT region, SUM(pib) as total_pib
FROM pays
GROUP BY region
ORDER BY total_pib DESC;
```

❑ **Exercice 3** : *Compression par codage par plages (RLE)*

Le **codage par plages** (*Run-Length Encoding* ou RLE) est une méthode classique de compression de données sans perte. Son principe consiste à remplacer les répétitions consécutives d'un même symbole par un couple formé de ce symbole et de son nombre d'apparitions successives.

En Python, le résultat compressé est représenté par une liste de couples `(caractere, compte)`, où `caractere` est de longueur 1 et `compte` est un entier strictement positif.

Par exemple, la chaîne `"AAABBCCCCA"` est encodée par la liste :

```
[('A', 3), ('B', 2), ('C', 4), ('A', 1)]
```

Q15– Donner le résultat du codage RLE de la chaîne `"WWWBWWWWBBBWW"`.

En supposant qu'un caractère et un entier occupent chacun un octet en mémoire, calculer la taille en octets de la chaîne initiale et celle de la liste encodée. Quel est le gain de compression obtenu ?

La chaîne comporte 3 fois `'W'`, 1 fois `'B'`, 4 fois `'W'`, 3 fois `'B'` et 2 fois `'W'`. Le résultat encodé est donc :

```
[('W', 3), ('B', 1), ('W', 4), ('B', 3), ('W', 2)]
```

La chaîne initiale contient 13 caractères, soit 13 octets. La représentation compressée comporte 5 couples de 2 octets, soit 10 octets au total. Le gain est de $13 - 10 = 3$ octets (soit un taux de réduction d'environ 23,1 %).

Q16– Donner un exemple de chaîne pour laquelle le codage RLE conduit à une augmentation de la taille des données (expansion au lieu d'une compression). Quel est le pire des cas pour cet algorithme ?

Toute chaîne ne comportant aucune répétition consécutive, par exemple `"ABCDE"` (ou `"WBWBWB"`), sera encodée sous la forme `[('A', 1), ('B', 1), ('C', 1), ('D', 1), ('E', 1)]`. Chaque caractère de 1 octet devenant un couple de 2 octets, la taille est alors doublée (taux d'expansion de 200 %). Le pire des cas correspond donc aux chaînes où deux caractères consécutifs sont toujours distincts.

Q17– Écrire en Python une fonction `rle_encode(texte)` qui prend en argument une chaîne de caractères `texte` et renvoie la liste de tuples `(caractere, compte)` correspondant à son encodage RLE. Si la chaîne est vide, la fonction renverra la liste vide `[]`.

```
def rle_encode(texte):
    if len(texte) == 0:
        return []
    res = []
    car_courant = texte[0]
    compte = 1
    for i in range(1, len(texte)):
        if texte[i] == car_courant:
            compte += 1
        else:
            res.append((car_courant, compte))
            car_courant = texte[i]
            compte = 1
    res.append((car_courant, compte))
    return res
```

Q18– Écrire en Python la fonction réciproque `rle_decode(liste_rle)` qui prend en argument une liste de couples `(caractere, compte)` et reconstruit la chaîne de caractères initiale.

On rappelle qu'en Python, l'opérateur $*$ entre une chaîne et un entier permet de répéter la chaîne (par exemple `"A" * 3` s'évalue en `"AAA"`).

```
def rle_decode(liste_rle):
    res = ""
    for car, compte in liste_rle:
        res += car * compte
    return res
```

Q19– On considère la boucle `for car, compte in liste_rle` de la fonction `rle_decode`. Donner un invariant de boucle permettant de prouver la correction partielle de cette fonction, et justifier brièvement pourquoi la terminaison est assurée.

- **Invariant :** À la fin de la k-ième itération (pour $0 \leq k \leq n$ avec $n = \text{len(liste_rle)}$), la variable `res` contient la chaîne de caractères issue du décodage des k premiers couples de `liste_rle`.
  - *Initialisation :* avant la boucle ($k = 0$), `res` vaut `""`, ce qui est bien le décodage d'une liste vide de couples.
  - *Conservation :* si la propriété est vraie au rang k, l'itération $k+1$ ajoute `car * compte` correspondant au $(k+1)$-ième couple, ce qui prolonge le décodage aux $k+1$ premiers couples.
  - *Terminaison :* en sortie de boucle ($k = n$), `res` correspond bien au décodage complet de toute la liste.
- **Terminaison :** La boucle parcourt une liste finie d'éléments à l'aide d'une instruction `for` en Python ; le nombre d'itérations est donc fini (égal à n), ce qui garantit la terminaison.

Q20– Écrire une fonction `longueur_totale(liste_rle)` qui calcule et renvoie la longueur totale de la chaîne décompressée sans reconstruire la chaîne en mémoire.

```
def longueur_totale(liste_rle):
    total = 0
    for _, compte in liste_rle:
        total += compte
    return total
```

Q21– Déterminer la complexité temporelle de la fonction `rle_encode(texte)` en fonction de la longueur N de la chaîne `texte`. Justifier la réponse.

La fonction parcourt la chaîne de longueur N au moyen d'une unique boucle `for` effectuant $N-1$ itérations. Chaque itération n'exécute que des opérations en temps constant $\mathcal{O}(1)$ (comparaisons, affectations, incrémentations et ajout en fin de liste avec `append`). La complexité temporelle globale est donc linéaire, en $\mathcal{O}(N)$.